Querying a Codebase with NVIDIA Auto Ontology

Will Cook

4 November 2026

A case study of code-ontology-bridge

Abstract

NVIDIA Auto Ontology's question-answering agent queried a relational map of its own Python source. It recovered a recursive caller chain that includes a callback, mutually dependent packages, and a base class with its recorded subclasses. The bridge extracts source facts and existing docstrings into eight tables, then supplies data and semantic metadata through Auto Ontology's existing interfaces. In an exploratory set of 22 author-written questions, the initial run produced ten complete normalized matches to reference rows. Replacing five responses with selected SQL-route reruns produced eleven automatic matches; two further identity checks gave thirteen matches, eight partial reference tables and one empty result. Eighteen questions included hints. The successful question run used a public-model compatibility patch and DeepSeek through OpenRouter. The experiment demonstrates the integration on one source snapshot. It does not measure an accuracy advantage over schema-only querying or other code-navigation tools.

1. The question this project explores

A developer investigating a change needs to know what calls a function, which packages depend on it and where relevant tests might be. Those questions combine descriptions with relationships spread across source files. A source map can store that information explicitly. A database question-answering agent can then turn a question into a query over the map.

This project connects those two approaches. NVIDIA Auto Ontology already provides a semantic layer and an agent that queries relational databases. CodeMap, an earlier tool developed by Will Cook, provides the starting idea of storing source descriptions and connections for people and coding agents to inspect. The bridge adapts that idea into tables and exports that Auto Ontology accepts.

The worked example asks Auto Ontology about its own implementation. One question asks for the direct and indirect callers of `resolve_semantic_fks`, excluding tests and giving the shortest call depth. Its agent generated recursive SQL and returned four callers:

Caller	Shortest depth
`_run_semantic_compilation`	1
`run_semantic_compilation`	2
`SemanticScheduler._run_once`	3
`main`	3

The scheduler passes the compilation function to `asyncio.to_thread`. A callback reference in the extracted map connects this caller to the rest of the chain. The returned rows agree with the recorded static graph and the inspected source. The request included guidance about reverse traversal and callbacks. The example establishes that the agent can query those represented relationships; it does not establish that the graph contains every possible runtime dependency. The saved Q11 record includes the request, generated SQL and rows.

2. What the two starting systems provide

Auto Ontology associates database tables and columns with semantic definitions. Terms identify entities; column attributes describe fields; SQL attributes describe calculations; semantic foreign keys describe joins. Connectors provide metadata and execute queries. A compilation process can use models, samples and query history to enrich the definitions. The question pipeline retrieves relevant context, generates SQL, checks aspects of the query, executes it and attempts repairs. A separate information route answers questions about catalog metadata. REST and the Model Context Protocol expose these services.

In this setting, an ontology is the implemented catalog and semantic model. It gives an agent descriptions and relationships to use when constructing a query. The data returned by that query still comes from the connected database.

CodeMap began as a way to inspect software written with coding agents. Descriptions stored under fixed headings in source docstrings could be read alongside static connections between files. People and agents could use the same saved account of the code. The public bridge carries that source-description approach into a standalone extractor; it does not require the original private interface.

The two systems meet at a concrete representation. A file or function becomes a row. A call relates one symbol row to another. Table descriptions and join definitions tell Auto Ontology how those records fit together. Auto Ontology supplies the question-answering machinery, while the bridge supplies the code-derived rows and their model.

3. The representation and its exports

The builder parses Python syntax without executing the analyzed program. Existing docstrings supply descriptions, with whitespace normalized. A recognized purpose section takes precedence; otherwise the extractor uses the first paragraph. Missing descriptions stay empty. No language model generates descriptions during extraction.

The recorded Auto Ontology map contains:

Table	Rows	Meaning
`snapshot`	1	Revision, source fingerprint and analysis limits
`files`	473	Paths, module descriptions and file attributes
`headings`	0	Recognized fixed docstring sections, when present
`symbols`	5,319	Functions, methods, classes and source locations
`class_bases`	306	Declared base-class references
`calls`	31,235	31,178 direct calls and 57 callback references
`imports`	4,762	Imports and resolvable file targets
`test_links`	483	Associations inferred from imports or filenames

The 473 files include 168 test files. Of the file descriptions, 399 come from module docstrings and 74 are empty. Auto Ontology's ordinary docstrings do not use CodeMap's fixed headings, so the headings table is empty. Two views summarize incoming calls and public symbols in files without a recorded test link. The snapshot summary records the counts and omissions.

Three exports serve different purposes. The DuckDB file contains the rows and views that Auto Ontology queries. The model YAML describes the catalog, terms, attributes and joins. The evaluation directory contains PostgreSQL DDL, CSV data, reference questions and SQL, and seven named analyses in NVIDIA's evaluation layout. Producing that directory is separate from executing the upstream benchmark harness, which was not run.

These data and import interfaces already existed in Auto Ontology. The bridge's YAML validates against its import schema. The YAML includes ten catalog foreign-key links and ten semantic foreign-key links, whereas the DuckDB connector itself reports no primary or foreign keys. Importing

this metadata does not establish an executable data connection. The live experiment ingested the database and remapped catalog identifiers before importing the corresponding model; fresh bridge identifiers otherwise conflicted with the already-ingested database identity.

4. What ran

The experiment used Auto Ontology's own ingestion service, FastAPI backend, semantic compiler and question agent. DuckDB held the source rows. PostgreSQL 17 with pgvector held the service's catalog and semantic state. The service ran natively rather than in Docker.

Public model access required additional configuration work. The original session reported that the configured default model returned HTTP 410; that individual capture was later overwritten. Surviving Nemotron logs contain structured-output rejections and timeouts. A nine-file patch changes the example model default, adds a reasoning setting, permits explicit model-client selection and exposes the existing `calculation_only` option through the chat API. The patch does not replace the query agent or repair compilation's success reporting.

The recorded Nemotron compilation took 663.3 seconds before the changes and 32.1 seconds with reasoning disabled. Several settings changed together, so these are observations about two configurations rather than a controlled speedup measurement. The before log contains 24 matching `guided_json` lines, including repeated error text; these are not 24 independently counted failed calls. The final question results below use DeepSeek, not the Nemotron timing configuration.

The patch has a historical report of 178 passing tests in the affected utility and chat modules, plus lint and formatting checks. That report is distinct from a new validation run and from the bridge's own tests. The patch documentation identifies the affected files and scope.

5. Evaluation protocol and results

The author supplied the extractor, the 22 questions and their reference SQL. Reference rows were computed over the extracted map. Questions cover descriptions, calls, imports, package relationships, inheritance and static test associations.

Eighteen questions include evidence hints. The request driver sends each question and hint to the chat API; it appends reference SQL and rows to the saved record after receiving the response. Seven named SQL analyses were imported into the semantic model, including a caller-traversal example overlapping Q11. Its backend log reports zero custom analyses in the query-construction context. An empty `custom_analyses_used` response field is not independent evidence of this: the upstream response code hardcodes that field to an empty list. No experiment removed the preloaded analyses to measure their effect.

After a three-question trial, the agent received all 22 questions using default routing. Five questions went to the information route and returned no SQL rows. Those five were rerun with `calculation_only` enabled. The combined result keeps the other seventeen original responses and substitutes the five rerun responses.

How rows were compared

The original scorer reports exact matches, normalized matches and partial reference projections. Exact matching preserves row and column order. Normalization ignores both orders and column names, compares strings case-insensitively, drops the path prefix of symbol identifiers and rounds numbers to one decimal place. The partial predicate accepts an equally sized result whose values are a subset of each corresponding reference row. A normalized match takes precedence over a partial result.

Stage	Normalized	Partial	Unmatched
Initial 22 responses	10	7	5
Five selected reruns	1	1	3

Stage	Normalized	Partial	Unmatched
Combined automatic result	11	8	3
Combined with identity checks	13	8	1

Five initial responses and six combined responses were exact matches; those are subsets of the normalized counts. Two combined responses need manual identity checks. Q15 returns dotted module names corresponding to the same twelve reference paths. Q17 returns names and files corresponding to the same seven reference symbol identifiers; its line ranges also match the source. These checks account for the change from eleven automatic matches to thirteen.

The eight partial tables omit different fields. Q2 omits a description-source label, Q6 omits a fingerprint, and several others omit counts or percentages. Some omissions concern fields the question did not explicitly request. Reference-row completeness and satisfaction of a natural-language question are therefore different judgments. The scoring concerns structured rows: Q16 has exact rows but a placeholder in its prose-response field. Natural-language answer quality was not systematically assessed.

Q4 remains empty after forced SQL routing. It asks for a function's location and docstring, but the generated query searches the empty headings table instead of the symbols table's summary. Routing repair therefore leaves a separate field-selection failure.

Other successful queries return fourteen package pairs with resolved calls in both directions and identify `SQLDatabase` with ten direct non-test subclasses under the question's recorded construction-call condition. All selected responses and their classifications are linked from the detailed results.

Scope of the result

The combined 13/8/1 outcome is a selected result after intervention, including two manual judgments. It is not one fresh run of 22 questions under a fixed policy. The normalized scorer is permissive: removing paths can collapse distinct identities, while ignoring column roles can hide a mismatch. Agreement on one database does not prove that two SQL queries are equivalent on other data.

The original session reported an OpenRouter account-usage increase of approximately US$0.54 across the trial, full pass and reruns. This excludes NVIDIA APIs and local compute, and its attribution assumes no unrelated account usage. A standalone billing record is not included in the public evidence. Cost is a reported engineering observation, not a measured cost-performance comparison.

6. Certification and answer annotations

The bridge also provides local governance commands. A certification event records a check, actor, source fingerprint, revision and decision. A planner converts eligible states into requests to Auto Ontology's existing certification routes. A separate command inspects returned SQL and associates candidate table and column references with definitions in a supplied model and their local certification states. Reconciliation assigns competing definitions using a fixed precedence among identifiers, names, explicit resolutions and aliases.

These additions have narrower evidence than the main query integration. The historical live demonstration used reference-SQL reproduction checks to gate certification flag updates. That establishes the path from a recorded check to a flag; it does not establish the correctness of the term text being flagged. The SQL annotation is post-hoc matching, not a trace of which definitions the agent retrieved or relied on. Certification state does not automatically restrict the upstream agent's queries or stay synchronized with later source changes.

The governance guide and capability tracker distinguish local behavior, historical live observations and remaining defects.

7. Limits and relation to earlier work

The map is a static approximation. Of 31,178 direct call sites, 9,442 have an internal callee identifier. A further 11,794 are classified as dynamic attribute calls without one. The small residual category called `unresolved` is therefore not a measure of graph completeness. The snapshot also counts 1,046 module-level calls omitted from the call table. Callback records have low confidence because passing a function does not prove invocation. Test associations are not executed test coverage.

Source identity also needs further engineering. A valid property getter and setter can share a qualified name, producing duplicate symbol primary keys in the current extractor. The demonstrated snapshot does not establish reliable export for every Python repository. This limitation and the governance defects are retained in GAPS rather than hidden by broader support claims.

Queryable code representations are established techniques. CodeQL builds databases for source analysis. CodexGraph lets language-model agents construct queries over code graph databases. The contribution here is a concrete adapter to Auto Ontology's existing contracts and an inspectable use of its agent on code-derived rows. It is not a priority claim for querying code as data.

The evaluation is also narrower than semantic SQL equivalence. Zhong, Yu and Klein (2020) use test suites of databases to assess query behavior across data instances. This experiment compares outputs on one map under its original scorer. It has no independent question set, no second repository, no schema-only control and no comparison with an agent reading source directly.

A useful next experiment would freeze routing, hints, model, source and scoring before collecting a complete fresh run. It would preserve typed columns and full symbol identities, score prose separately, and compare the same data and model with and without semantic context. Independently authored questions would help test whether the adapter generalizes beyond the tasks used to develop it. Those measurements remain future work.

8. Reproduction and evidence

The analyzed Auto Ontology revision is `cdaf61e1bd0cf3820d11d72b43bb460816aebbd3`. The original bridge revision is `b8ed6f201bc6c2c2e402ba39169b71e0acb99250`. The final chat model identifier is `deepseek/deepseek-v4.1-flash-1017`. These identify the historical experiment, not a claim about current public defaults.

The reproduction guide separates a local build, optional compatibility tests and live service replay. The repository contains source, reference queries, exported data and saved answers. Editorial corrections preserve those original records. Several early import and certification captures were overwritten during development; the detailed results label those observations as historical reports rather than complete standalone captures.

The demonstrated result is practical: Auto Ontology can use a source map to answer structural questions about the implementation from which it was built. The saved queries explain how it did so, and the separated run stages show where configuration, routing and scoring affect that conclusion.